

Lab 3 — Exercise: Wrangling the Google Play Store Dataset

ITCS 3162 — Introduction to Data Mining

Name: Srinivas Dengle Date: 05/31/2026

This dataset is *messier* than diamonds — it's scraped from the Google Play Store and has the kinds of problems you'll see in the wild: numbers stored as strings, units mixed with values, duplicates, weird placeholder strings, and missing values. That's the point: you'll do real cleaning.

Source: Kaggle's "Google Play Store Apps" dataset (mirrored on GitHub for stable URL access).

When you're done, **Restart & Run All**, download as `.ipynb`, and submit via Canvas.

Setup

Run this cell. It loads the dataset directly from a GitHub URL.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

URL = "https://raw.githubusercontent.com/sumitgirwal/google-play-store-data-analysis/master/googleplaystore.csv"
df = pd.read_csv(URL)
df.head()
```

	App	Category	Rating	Reviews	Size	Installs	Type	Price	Content Rating	Genres	Last Updated	Current Ver	Android Ver	
0	Photo Editor & Candy Camera & Grid & ScrapBook	ART_AND_DESIGN	4.1	159	19M	10,000+	Free	0	Everyone	Art & Design	January 7, 2018	1.0.0	4.0.3 and up	
1	Coloring book moana	ART_AND_DESIGN	3.9	967	14M	500,000+	Free	0	Everyone	Art & Design;Pretend Play	January 15, 2018	2.0.0	4.0.3 and up	
2	U Launcher Lite – FREE Live Cool Themes, Hide ...	ART_AND_DESIGN	4.7	87510	8.7M	5,000,000+	Free	0	Everyone	Art & Design	August 1, 2018	1.2.4	4.0.3 and up	

Next steps: [Generate code with df](#) [New interactive sheet](#)

Exercise 1 — Initial inspection (10 pts)

In the cell below, print **all three**:

- 1. The shape of the DataFrame (rows, columns)
- 2. The output of `df.info()`
- 3. The output of `df.describe(include="all")`

Then in the markdown cell, answer the questions.

```
# TODO: print shape, info(), describe(include='all')
print(df.shape)
print("")

df.info()

display(df.describe(include="all"))
```

```
(10841, 13)
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10841 entries, 0 to 10840
Data columns (total 13 columns):
#   Column          Non-Null Count  Dtype
---  -
0   App              10841 non-null  object
1   Category         10841 non-null  object
2   Rating           9367 non-null   float64
3   Reviews          10841 non-null  object
4   Size             10841 non-null  object
5   Installs         10841 non-null  object
6   Type             10840 non-null  object
7   Price            10841 non-null  object
8   Content Rating   10840 non-null  object
9   Genres           10841 non-null  object
10  Last Updated     10841 non-null  object
11  Current Ver      10833 non-null  object
12  Android Ver      10838 non-null  object
dtypes: float64(1), object(12)
memory usage: 1.1+ MB
```

	App	Category	Rating	Reviews	Size	Installs	Type	Price	Content	Rating	Genres	Last Updated	Current Ver	Android Ver
count	10841	10841	9367.000000	10841	10841	10841	10840	10841		10840	10841	10841	10833	10838
unique	9660	34	NaN	6002	462	22	3	93		6	120	1378	2832	33
top	ROBLOX	FAMILY	NaN	0	Varies with device	1,000,000+	Free	0		Everyone	Tools	August 3, 2018	Varies with device	4.1 and up
freq	9	1972	NaN	596	1695	1579	10039	10040		8714	842	326	1459	2451
mean	NaN	NaN	4.193338	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
std	NaN	NaN	0.537431	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
min	NaN	NaN	1.000000	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
25%	NaN	NaN	4.000000	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
50%	NaN	NaN	4.300000	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
75%	NaN	NaN	4.500000	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
max	NaN	NaN	19.000000	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN

Answer the following:

1. How many rows and columns does the dataset have?
2. Look at the `Reviews`, `Size`, `Installs`, and `Price` columns. What dtype does pandas assign them? Why is that a problem for a numeric column like `Reviews`?
3. Which columns appear to have missing values?

YOUR ANSWERS: \

1. (10841,13)
2. It is assigned object
3. Rating, type, content rating, current ver, android ver

✓ Exercise 2 — Missing values (10 pts)

In the cell below, compute (a) the count of missing values per column, sorted descending, and (b) the **percentage** of missing values per column rounded to 2 decimal places. Then drop any row that is missing the `Rating` column (since it's the most important missingness to handle) and store the result in `df_r`.

```
# TODO: counts of missing values per column (sorted descending)
missing_counts = df.isnull().sum().sort_values(ascending=False)
print(missing_counts)

# TODO: percentage missing per column (rounded to 2 decimal places)
missing_percentage = (df.isnull().sum() / len(df) * 100).round(2).sort_values(ascending=False)
print("")
print("Missing Percentage")
print(missing_percentage)

# TODO: create df_r by dropping rows where Rating is NaN; print its new shape
print("")
df_r = df.dropna(subset=["Rating"])
print(f"New DataFrame shape: {df_r.shape}")
```

```
Rating      1474
Current Ver      8
Android Ver      3
Content Rating    1
Type            1
Size            0
Reviews         0
Category        0
App             0
Price           0
Installs        0
Last Updated     0
Genres          0
dtype: int64

Missing Percentage
Rating      13.60
Current Ver   0.07
Android Ver   0.03
Content Rating 0.01
Type          0.01
Size          0.00
Reviews       0.00
Category      0.00
App           0.00
Price         0.00
Installs      0.00
```

```
Last Updated    0.00
Genres          0.00
dtype: float64

New DataFrame shape: (9367, 13)
```

Exercise 3 — Duplicates (10 pts)

There are duplicate rows in this dataset (the same app appears multiple times). In the cell below:

1. Print how many fully-duplicate rows exist in `df_r`.
2. How many duplicate values are there in the `App` column specifically (apps appearing under different categories)?
3. Create `df_dedup` by dropping fully-duplicate rows and print its new shape.

```
# TODO: fully duplicate
fully_duplicates = df_r.duplicated().sum()
print(f"Number of fully-duplicate rows: {fully_duplicates}")

# TODO: duplicate App names (use df_r["App"].duplicated().sum())
app_duplicates = df_r["App"].duplicated().sum()
print(f"Number of duplicate App names: {app_duplicates}")

# TODO: df_dedup = df_r.drop_duplicates(); print shape
df_dedup = df_r.drop_duplicates()
print(f"Shape of df_dedup after removing full duplicates: {df_dedup.shape}")
```

```
Number of fully-duplicate rows: 474
Number of duplicate App names: 1170
Shape of df_dedup after removing full duplicates: (8893, 13)
```

Exercise 4 — Cleaning the `Installs` column (15 pts)

The `Installs` column looks like `"1,000,000+"` — a string with a `+` and commas. Convert it to an integer column. Steps:

1. Inspect a few unique values: `df_dedup["Installs"].unique()[:10]`
2. Remove `+` and `,` characters with `.str.replace()`
3. Convert to int with `.astype(int)` (or use `pd.to_numeric` with `errors="coerce"` if any values look weird)
4. Assign the cleaned values back to `df_dedup["Installs"]`
5. Verify with `df_dedup["Installs"].dtype` and `.head()`

```
# TODO: inspect unique values
print("Sample of raw unique Installs values:")
print(df_dedup["Installs"].unique()[:10])

df_dedup["Installs"] = df_dedup["Installs"].str.replace("+", "", regex=False)
df_dedup["Installs"] = df_dedup["Installs"].str.replace(",", "", regex=False)

df_dedup["Installs"] = pd.to_numeric(df_dedup["Installs"], errors="coerce")
```

```
# TODO: clean Installs to integer
print(f"Cleaned Installs column dtype: {df_dedup['Installs'].dtype}")
print("\nFirst few rows of cleaned Installs:")
print(df_dedup["Installs"].head())
```

Sample of raw unique Installs values:
 ['10,000+' '500,000+' '5,000,000+' '50,000,000+' '100,000+' '50,000+'
 '1,000,000+' '10,000,000+' '5,000+' '100,000,000+']
 Cleaned Installs column dtype: float64

```
First few rows of cleaned Installs:
0      10000.0
1     500000.0
2    5000000.0
3   50000000.0
4     100000.0
Name: Installs, dtype: float64
/tmp/ipykernel_1627/1119460307.py:5: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead
```

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
 df_dedup["Installs"] = df_dedup["Installs"].str.replace("+", "", regex=False)
 /tmp/ipykernel_1627/1119460307.py:6: SettingWithCopyWarning:
 A value is trying to be set on a copy of a slice from a DataFrame.
 Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
 df_dedup["Installs"] = df_dedup["Installs"].str.replace("+", "", regex=False)
 /tmp/ipykernel_1627/1119460307.py:8: SettingWithCopyWarning:
 A value is trying to be set on a copy of a slice from a DataFrame.
 Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
 df_dedup["Installs"] = pd.to_numeric(df_dedup["Installs"], errors="coerce")

✓ Exercise 5 — Cleaning the Price column (15 pts)

Price looks like "0" for free apps and "\$4.99" for paid ones. Convert it to a float.

1. Use `.str.replace("$", "", regex=False)` then `.astype(float)`.
2. Watch for any unexpected values (the real dataset has at least one weird row — the column may have a non-numeric placeholder that `pd.to_numeric(..., errors="coerce")` handles gracefully).
3. After cleaning, print how many paid apps there are (where `Price > 0`).

```
# TODO: clean Price to float
df_dedup["Price"] = df_dedup["Price"].str.replace("$", "", regex=False)
df_dedup["Price"] = pd.to_numeric(df_dedup["Price"], errors="coerce")
```

```
# TODO: count paid apps
paid_apps_count = (df_dedup["Price"] > 0).sum()
print(f"Number of paid apps in the dataset: {paid_apps_count}")
```

```
Number of paid apps in the dataset: 613
/tmp/ipykernel_1627/2739529782.py:2: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead
```

```
See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user\_guide/indexing.html#returning-a-view-versus-a-copy
df_dedup["Price"] = df_dedup["Price"].str.replace("$", "", regex=False)
/tmp/ipykernel_1627/2739529782.py:3: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead
```

```
See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user\_guide/indexing.html#returning-a-view-versus-a-copy
df_dedup["Price"] = pd.to_numeric(df_dedup["Price"], errors="coerce")
```

✓ Exercise 6 — Filtering and a derived column (15 pts)

1. Create `top_apps`, containing only apps with `Rating >= 4.5` and `Installs >= 1_000_000`.
2. Print its shape and the top 5 categories of those apps by count.
3. In `df_dedup`, add a new column `Revenue` defined as `Price * Installs` (zero for free apps).
4. Show the top 10 apps by `Revenue`.

```
# TODO: top_apps and category counts
top_apps = df_dedup[(df_dedup["Rating"] >= 4.5) & (df_dedup["Installs"] >= 1000000)]

print("")
print(f"Shape of top_apps DataFrame: {top_apps.shape}")
print("\nTop 5 app categories in top_apps:")
print(top_apps["Category"].value_counts().head(5))
df_dedup["Revenue"] = df_dedup["Price"] * df_dedup["Installs"]
print("")

# TODO: Revenue column and top 10
print("Top 10 apps by Revenue:")
display(df_dedup[["App", "Category", "Price", "Installs", "Revenue"]].sort_values(by="Revenue", ascending=False).head(10))
```

Shape of top_apps DataFrame: (1234, 13)

Top 5 app categories in top_apps:

```
Category
GAME          272
FAMILY        171
TOOLS          89
HEALTH_AND_FITNESS 82
PRODUCTIVITY  62
Name: count, dtype: int64
```

Top 10 apps by Revenue:

/tmp/ipykernel_1627/3691241148.py:8: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
df_dedup["Revenue"] = df_dedup["Price"] * df_dedup["Installs"]
```

	App	Category	Price	Installs	Revenue
4347	Minecraft	FAMILY	6.99	10000000.0	69900000.0
2241	Minecraft	FAMILY	6.99	10000000.0	69900000.0
5351	I am rich	LIFESTYLE	399.99	100000.0	39999000.0
5356	I Am Rich Premium	FINANCE	399.99	50000.0	19999500.0
4034	Hitman Sniper	GAME	0.99	10000000.0	9900000.0
7417	Grand Theft Auto: San Andreas	GAME	6.99	1000000.0	6990000.0
2883	Facetune - For Free	PHOTOGRAPHY	5.99	1000000.0	5990000.0
5578	Sleep as Android Unlock	LIFESTYLE	5.99	1000000.0	5990000.0
8804	DraStic DS Emulator	GAME	4.99	1000000.0	4990000.0
4367	I'm Rich - Trump Edition	LIFESTYLE	400.00	10000.0	4000000.0



Exercise 7 — Group-by summary (15 pts)

Using `df_dedup`, produce a single summary table with one row per **Category**, containing:

- `n_apps` — number of apps in that category
- `mean_rating` — average rating (rounded to 2 decimals)
- `mean_installs` — average installs (rounded to 0 decimals)
- `total_revenue` — sum of `Revenue`

Sort by `total_revenue` descending. Show only the top 10 rows.

```
# TODO: groupby summary
category_summary = (
    df_dedup.groupby("Category")
    .agg(
        n_apps=("App", "count"),
        mean_rating=("Rating", "mean"),
```

```

        mean_installs=("Installs", "mean"),
        total_revenue=("Revenue", "sum"),
    )
    .round({"mean_rating": 2, "mean_installs": 0})
)

display(category_summary.sort_values(by="total_revenue", ascending=False).head(10))

```

	n_apps	mean_rating	mean_installs	total_revenue	
Category					
FAMILY	1718	4.19	5844663.0	1.857743e+08	
LIFESTYLE	305	4.10	1753250.0	5.758394e+07	
GAME	1074	4.28	29370449.0	4.098684e+07	
FINANCE	317	4.13	2430008.0	2.572664e+07	
PHOTOGRAPHY	304	4.18	31977773.0	8.941050e+06	
MEDICAL	302	4.18	139612.0	8.371355e+06	
PERSONALIZATION	310	4.33	6691461.0	7.786310e+06	
TOOLS	734	4.05	15600442.0	5.462910e+06	
SPORTS	286	4.23	5344516.0	4.706154e+06	
PRODUCTIVITY	334	4.20	37314581.0	4.304452e+06	

Exercise 8 — Reflection (10 pts)

In 4–6 sentences, answer all of:

1. Which cleaning step would have been impossible to skip if you wanted to compute average install counts? Why?
2. What's one piece of information in the raw data that you *could* extract with more work but didn't (e.g., the `Size` column, the `Last Updated` column)? How would you approach it?
3. Did anything in the data surprise you?

YOUR ANSWER:

Cleaning the Installs column was non-negotiable — commas and plus signs caused Pandas to read it as plain text, making any numerical aggregation impossible. The Size column would've been another fun challenge, since it mixes units like "M" and "k" alongside placeholders like "Varies with device"; the fix would involve standardizing everything to bytes through unit mapping and type casting. Perhaps the most striking takeaway was the revenue paradox: despite the overwhelming majority of apps being free, a handful of paid apps still manage to pull in tens of millions from upfront purchases alone.

Submission checklist

- ☐ Name and date filled in
- ☐ All TODO cells completed and run
- ☐ All `YOUR ANSWER` prompts replaced

- ☐ **Restart & Run All** completes without errors